

Protected Variations

Protected Variations Principle

Wrap what varies behind a stable interface

DEFINITION

Identify points of predicted variation or instability and wrap them behind a stable interface.

Protected Variations is the principle the others serve: find the points where change is likely — a vendor, a data format, a platform, a volatile module — and wrap each behind a stable interface. Clients depend only on that contract, so when the implementation behind it changes, nothing on the other side has to. Larman calls it the root principle that Low Coupling, Polymorphism, and Indirection all work toward.

WHY IT MATTERS

Almost all expensive change is change that leaks across a boundary — a swapped payment provider forcing edits in unrelated code. A stable interface acts as a firewall: the variation is contained at the seam and clients stay untouched. The discipline is to protect predicted variation, not imagined variation — guarding every "what if" is speculative generality, which costs more than it saves.

— How it works

Variation point

A spot where change is predicted or instability is likely — vendor, format, protocol, platform.

Stable interface

The contract clients depend on; it stays put while implementations move behind it.

Implementations

The interchangeable concretes hidden behind the interface and swapped without client edits.

HOW TO APPLY

1. Identify what is likely to change or is already unstable.
2. Define a stable interface that expresses what clients need, not how it's done.
3. Put each implementation behind that interface.
4. Depend only on the interface — and resist wrapping changes you can't actually foresee (YAGNI).

LITMUS TEST

Is this a known or likely change (vendor, format, protocol)? If yes, wrap it. Is it speculative? Then YAGNI — protect predicted variation, not imagined variation.

— Smell → fix

Business code calls Stripe directly — switching or adding a provider means editing every call site:

```
class Checkout {  
  pay(cents: number) {  
    new Stripe(key).charge(cents) // wired to one vendor everywhere  
  }  
}
```

↓ WRAP THE VARIATION

```
interface PaymentGateway { pay(cents: number): Promise }  
class StripeGateway implements PaymentGateway {  
  pay(c: number) { return stripe.charge(c) }  
}  
class Checkout {  
  private gw!: PaymentGateway // the stable seam, injected  
}
```

Stripe, PayPal, or a mock all sit behind PaymentGateway; clients depend only on the contract. Guess the seam right and change is a new class — guess wrong and the abstraction is wasted cost.

USE WHEN

- ✓ A known / likely change — vendor, format, platform
- ✓ Shielding clients from churn

AVOID WHEN

- ▲ Protecting against speculative change — YAGNI; protect predicted variation

— Trade-offs & pitfalls

PROS

- + Change is contained at the seam
- + Pluggable, future-proof

CONS

- Over-protection adds cost
- Guessing the wrong seam

COMMON PITFALLS

- ▲ Speculative generality — protecting against imagined change you never end up needing.
- ▲ Guessing the seam wrong, so the "stable" interface leaks and churns anyway.
- ▲ Abstracting a single, never-changing implementation — cost with no protection earned.

WHERE YOU SEE IT

- A PaymentGateway interface with Stripe / PayPal implementations behind it.
- A Repository hiding the database; an ILogger / IClock hiding infrastructure.
- Adapters around third-party SDKs so a vendor swap touches one class.

SEE ALSO

OCP	the same idea from SOLID — open to new variants, closed to client edits
DIP	depend on the stable abstraction, not the volatile implementation
Indirection	a common mechanism for building the protective seam
Polymorphism	implementations behind the interface are selected polymorphically

— Interview & sources

Protected Variations vs OCP?

The same idea from two lineages: OCP (SOLID) and Protected Variations (GRASP) both isolate variation behind a stable abstraction. PV is the more general root principle.

How is PV different from speculative generality?

PV protects predicted, evidenced variation; speculative generality guards against imagined change. The discipline is YAGNI — wrap a seam only when the change is likely.

Why is PV called a "root" principle?

Larman frames Low Coupling, Polymorphism, and Indirection as ways to achieve it. Many patterns (Adapter, Strategy, Facade) are concrete Protected-Variations mechanisms.

How do you decide where to apply it?

Protect variation you can predict with evidence — a roadmap item, a flaky vendor, a format that's changed before. If you can't point to a concrete reason it will change, leave it concrete; YAGNI beats guessing.

REMEMBER

Find what will change, and hide it behind something that will not.

SOURCES

Craig Larman — "Applying UML and Patterns" (3rd ed., 2004): Protected Variations

David Parnas — "On the Criteria To Be Used in Decomposing Systems into Modules" (1972): information hiding